SMART MEDICINE SORTING & EXPIRY ALERT SYSTEM (SMSES)

An Arduino-Based FEFO Inventory Management System
Using Min-Heap and Circular Doubly Linked List.

A course project report submitted in partial fulfilment of the requirement

of

SMART SYSTEM DESIGN

By

Devaraneni Shanvika Patel

Under the guidance of

Mr. Y. Srikanth,

Asst. Prof., Department of ECE

Ms. Bhavana

Asst. Prof., Department of ECE



ABSTRACT

The Smart Medicine Sorting & Expiry Alert System (SMSES) is an Arduino-based prototype designed to organize medicine inventory and prioritize medicines according to their expiry dates. The system implements the First-Expired, First-Out (FEFO) principle to identify medicines with the earliest expiry dates and assist users in managing their inventory efficiently.

The core software architecture integrates a Min-Heap Priority Queue and a Circular Doubly Linked List (CDLL). The Min-Heap maintains expiry-based priority, providing efficient insertion and retrieval of the earliest-expiring medicine. The CDLL manages circular slot navigation and supports rotation when multiple medicines share the same expiry date. An array-based structure stores medicine information, including batch number, medicine name, expiry date, assigned slot and active status.

An Arduino Uno acts as the central controller and processes medicine information received through serial communication. The prototype supports four physical medicine slots, with Green LEDs indicating slot occupancy, Red LEDs identifying the highest-priority medicine and a White LED indicating system power. DISPENSE and NEXT push buttons provide user interaction, while a buzzer generates confirmation and near-expiry alerts.

The system demonstrates the practical application of fundamental data structures in embedded systems using a low-cost hardware platform. It provides an educational prototype for FEFO-based inventory management and offers opportunities for future enhancement through barcode scanning, real-time clock integration, automated dispensing and remote monitoring.

CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	2
1	**INTRODUCTION**	5
1.1	About the Project	5
1.2	Objectives	5
2	**PROJECT DESCRIPTION**	7
2.1	Block Diagram of the Project	7
2.2	Description of the Block Diagram	7
2.3	Hardware Description	8
2.3.1	Arduino UNO	8
2.3.2	LEDs (Red, Green and White)	9
2.3.3	Push Buttons	10
2.3.4	Buzzer	11
2.3.5	Arduino UNO (Main Controller)	11
2.3.6	Breadboard and Jumper Wires	12
2.3.7	Medicine Storage Tray (Four-Slot Arrangement)	13
2.4	Software Description	14
2.4.1	Arduino IDE	14
2.4.2	Program Structure	15
2.4.3	Data Structures in Software	16
2.4.4	Core Logic Modules	18
2.4.5	Timing and Real-Time Execution	19
2.4.6	Memory Optimization	21
2.4.7	Testing and Verification	21

3	**PROJECT IMPLEMENTATION**	23
3.1	Working	23
3.2	Results	26
3.3	Advantages	27
3.4	Disadvantages	28
4	**CONCLUSION**	29
4.1	Conclusion	29
4.2	Future Scope	29
	BIBLIOGRAPHY	31
	APPENDIX	32
A	Hardware Pin Configuration	32
B	Medicine Data Input Format	32
C	Serial Monitor Commands	33
D	Data Structures Used	33
E	Arduino Source Code	33

CHAPTER - 1

INTRODUCTION

1.1 ABOUT THE PROJECT

Medicines are essential for maintaining health, but managing their expiry dates can be challenging in households, medical stores, and healthcare facilities. Poor inventory management and a lack of regular expiry-date monitoring may result in medicines remaining unused beyond their expiry dates, leading to unnecessary wastage and potential safety concerns. Therefore, an organized system is required to monitor medicine inventory and prioritize medicines according to their expiry dates.

The **Smart Medicine Sorting & Expiry Alert System (SMSES)** is an Arduino-based prototype developed to demonstrate an efficient approach to medicine inventory management using the **First-Expired, First-Out (FEFO)** principle. The system accepts medicine details, including the batch number, medicine name, and expiry date, through serial communication. It processes this information and identifies the medicine with the earliest expiry date.

The system integrates a **Min-Heap Priority Queue** to maintain expiry-based priority and a **Circular Doubly Linked List (CDLL)** to support circular slot navigation. An array-based structure is also used to maintain medicine records and slot information. Four medicine slots are represented using Green and Red LED indicators, while a White LED indicates the system's power status. Two push buttons, DISPENSE and NEXT, provide user interaction, and a buzzer generates audio feedback for selected system operations and alerts.

By combining fundamental data structures with embedded system programming, SMSES demonstrates how software algorithms can be implemented using low-cost hardware. The project serves as an educational prototype for understanding FEFO-based inventory management and the practical application of data structures in real-world embedded systems.

1.2 OBJECTIVES

The main objectives of the Smart Medicine Sorting & Expiry Alert System are:

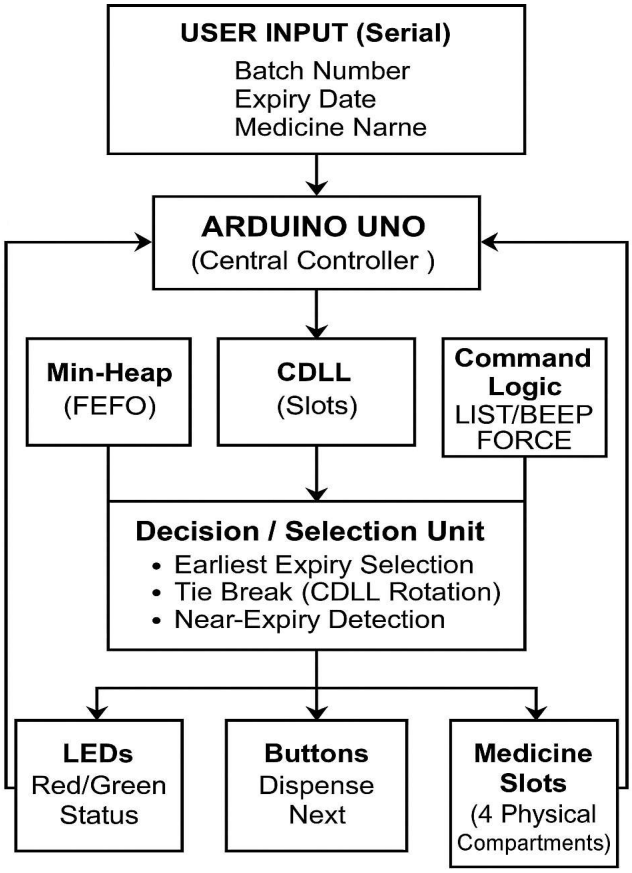
1. To develop an Arduino-based medicine inventory management prototype using the FEFO (First-Expired, First-Out) principle.
2. To implement a **Min-Heap Priority Queue** for prioritizing medicines based on their expiry dates.
3. To use a **Circular Doubly Linked List (CDLL)** for circular slot navigation and handling medicines with equal expiry dates.
4. To store and manage medicine information, including batch number, medicine name, expiry date, and slot details.
5. To identify the earliest-expiring medicine and indicate its priority using Red LEDs.

6. To display slot occupancy using Green LEDs and indicate the system's power status using a White LED.
7. To provide user interaction through DISPENSE and NEXT push buttons.
8. To generate audio feedback through a buzzer for user actions and selected alert conditions.
9. To demonstrate the integration of data structures, embedded C/C++ programming, and Arduino hardware in a low-cost prototype.
10. To provide a foundation for future enhancements such as barcode scanning, real-time clock integration, and automated medicine dispensing.

CHAPTER -2

PROJECT DESCRIPTION

2.1 BLOCK DIAGRAM OF THE PROJECT



SMART BIOCK DIAGRAM

2.2 Description of the Block Diagram

The Smart Medicine Sorting & Expiry Alert System is designed around the Arduino UNO, which acts as the central controller of the prototype. It receives medicine information through serial communication. The input includes the batch number, medicine name, and expiry date.

After receiving the input, the Arduino processes the medicine details and stores the records using an array-based structure. A Min-Heap Priority Queue is implemented to organize medicines according to their expiry dates. The medicine with the earliest expiry date is assigned the highest priority, following the FEFO (First-Expired, First-Out) principle.

A Circular Doubly Linked List (CDLL) is used to support circular navigation between the medicine slots. It also helps manage the rotation of items when medicines have equal expiry dates. The combination of these data structures enables organized inventory management and efficient priority identification.

The hardware components are connected to the Arduino UNO to provide visual and audio feedback. Four Green LEDs indicate the occupancy status of the four medicine slots, while

four Red LEDs identify the slot containing the highest-priority medicine. A White LED indicates that the system is powered ON.

Two push buttons are provided for user interaction. The DISPENSE button removes the highest-priority medicine from the system, while the NEXT button allows the user to navigate through the circular slot arrangement. A buzzer provides audio feedback for operations and configured alert conditions.

Thus, the block diagram represents the integration of input processing, data structure implementation, priority management, and hardware-based output indicators in the SMSES prototype.

2.3 HARDWARE DESCRIPTION

2.3.1 Arduino UNO

The Arduino UNO is the main microcontroller board used in the Smart Medicine Sorting & Expiry Alert System (SMSES). It is based on the ATmega328P microcontroller and acts as the central processing unit responsible for executing the program and controlling the connected hardware components.

The Arduino UNO receives medicine information through serial communication and processes the input using the implemented data structures. It manages medicine records, maintains expiry-based priority using a Min-Heap Priority Queue, and supports circular navigation through a Circular Doubly Linked List (CDLL).

Important Features:

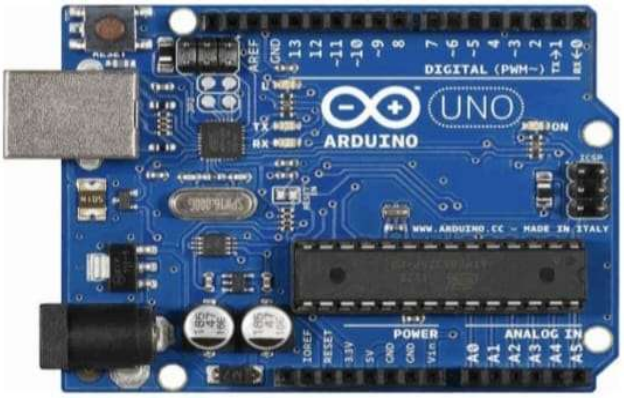
- **Microcontroller:** ATmega328P.
- **Operating Voltage:** 5V.
- **Digital Input/Output Pins:** 14 digital pins.
- **Analog Input Pins:** 6 analog pins.
- **Clock Speed:** 16 MHz.
- **Communication:** USB and serial communication.
- **Programming Environment:** Arduino IDE.

Role in the SMSES Project:

1. Receives medicine details through the Serial Monitor.
2. Processes batch numbers, medicine names and expiry dates.
3. Maintains medicine records and manages available slots.
4. Executes Min-Heap operations to identify the earliest-expiring medicine.
5. Controls Green, Red and White LED indicators.

6. Reads inputs from the DISPENSE and NEXT push buttons.
7. Controls the buzzer for audio feedback and configured alerts.

The Arduino UNO was selected because of its affordability, ease of programming, sufficient input/output pins and suitability for low-cost embedded system prototypes.



2.3.2 LEDs (Red/Green/White)

Light Emitting Diodes (LEDs) are used in the SMSES prototype to provide visual indications of medicine slot occupancy, FEFO priority and system power status. A total of nine LEDs are used in the project.

1. Green LEDs – Slot Occupancy Indicators

Four Green LEDs are connected to the four medicine slots.

- Each Green LED represents one medicine slot.
- It turns ON when a medicine is successfully assigned to that slot.
- It turns OFF when the corresponding medicine is removed or dispensed.

2. Red LEDs – FEFO Priority Indicators

Four Red LEDs are used to identify the medicine with the highest expiry-based priority.

- Each Red LED corresponds to one medicine slot.
- The Red LED associated with the earliest-expiring medicine turns ON.
- After a medicine is dispensed, the system updates the priority indication.

3. White LED – Power Indicator

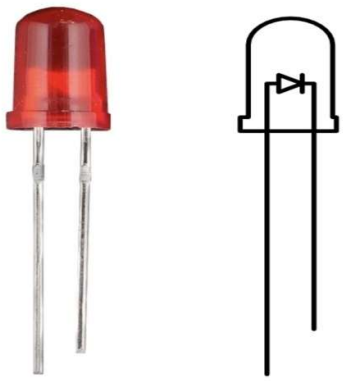
One White LED indicates the power status of the system.

- It turns ON when the system is powered.
- It provides a visual indication that the prototype is active.

Electrical Connections:

- Each LED is connected with a 220Ω current-limiting resistor.
- The LED anodes are connected to the assigned Arduino pins through resistors.
- The cathodes are connected to the common Ground (GND) rail.

These LED indicators provide a simple and effective way to monitor the system without requiring a separate display.



2.3.3 Push Buttons

Push buttons are input devices used to provide manual commands to the Smart Medicine Sorting and Expiry Alert System (SMSES). Two push buttons are used in the prototype: the DISPENSE button and the NEXT button.

1. DISPENSE Button

The DISPENSE button is connected to digital pin D6 of the Arduino UNO. When pressed, it removes the medicine with the earliest expiry date from the system's inventory using the Min-Heap data structure. The corresponding slot information is updated, and the Green and Red LEDs are refreshed to reflect the new inventory and expiry priority.

2. NEXT Button

The NEXT button is connected to digital pin D8 of the Arduino UNO. It is used to navigate through the medicine slots in a circular sequence. The Circular Doubly Linked List (CDLL) supports this navigation by moving to the next node and returning to the beginning after reaching the last node.

Working Principle

Both push buttons are connected between their respective Arduino input pins and GND. The Arduino uses the INPUT_PULLUP configuration to detect button presses. In this configuration, the input reads HIGH when the button is not pressed and LOW when it is pressed.

The buttons provide a simple way for users to interact with the system without requiring a separate display or complicated input interface.

2.3.4 Buzzer

A buzzer is an output device used to generate audible alerts in the Smart Medicine Sorting and Expiry Alert System (SMSES). It helps notify the user about important system events through sound signals.

In this project, a buzzer is connected to digital pin D7 of the Arduino UNO. The Arduino controls the buzzer based on the conditions defined in the program.

Functions of the Buzzer

Expiry Alert: Generates an alert when the programmed near-expiry condition is detected.

Dispense Confirmation: Produces a short beep when the user presses the DISPENSE button.

Navigation Confirmation: Provides an audible indication when the NEXT button is pressed.

Error Alert: Generates an alert when the user attempts to dispense medicine while the inventory is empty.

Working Principle

The buzzer receives a control signal from the Arduino UNO through digital pin D7. Based on the programmed conditions, the Arduino activates the buzzer to produce different audible indications.

The buzzer improves the user experience by providing immediate audio feedback without requiring a display screen. It also helps draw attention to important alerts during system operation.



2.3.5 Arduino UNO (Main Controller)

The Arduino UNO acts as the main controller of the Smart Medicine Sorting and Expiry Alert System (SMSES). It coordinates the hardware components and executes the programmed logic required for medicine inventory management and expiry-based prioritization.

The Arduino receives medicine details through serial communication and processes the information using the programmed data structures and algorithms. It manages the medicine

records, identifies the medicine with the earliest expiry date, and controls the output devices according to the system's operating conditions.

Functions of the Main Controller

1. **Data Input:** Receives medicine information, including batch number, medicine name, and expiry date, through the Serial Monitor.
2. **Data Processing:** Processes the entered information and stores the medicine records in the system.
3. **Priority Management:** Uses a Min-Heap to identify the medicine with the earliest expiry date based on the FEFO principle.
4. **Slot Navigation:** Uses a Circular Doubly Linked List (CDLL) to support circular navigation between medicine slots.
5. **Input Handling:** Detects user commands through the DISPENSE and NEXT push buttons.
6. **LED Control:** Controls the Green LEDs to indicate occupied slots, Red LEDs to indicate expiry priority, and the White LED to indicate power status.
7. **Audio Alerts:** Controls the buzzer to provide confirmation sounds and programmed alerts.

Working Principle

When medicine details are entered through the Serial Monitor, the Arduino processes and stores the information. The Min-Heap organizes the medicine records according to their expiry dates. When the DISPENSE button is pressed, the system logically removes the highest-priority medicine from the inventory and updates the corresponding indicators.

The NEXT button allows the user to navigate through the medicine slots using the CDLL. The Arduino continuously handles user inputs and updates the connected output devices according to the programmed logic.

Thus, the Arduino UNO serves as the central processing and control unit, integrating the hardware components with the software-based data structures to implement the FEFO-based medicine management system.

2.3.6 Breadboard and Jumper Wires

A breadboard and jumper wires are used to establish electrical connections between the Arduino UNO and the various components of the Smart Medicine Sorting and Expiry Alert System (SMSES). They make it possible to build and test the circuit without permanently soldering the components.

1. Breadboard

A breadboard is a reusable platform used for constructing electronic circuits. It contains interconnected holes that allow components and wires to be inserted easily.

In this project, the breadboard is used to:

- Connect the LEDs, push buttons and buzzer to the circuit.
- Distribute power and Ground (GND) connections.
- Establish temporary connections for testing and debugging.
- Make circuit modifications easier during development.

2. Jumper Wires

Jumper wires are insulated wires used to connect different components of an electronic circuit.

In this project, jumper wires are used to:

- Connect the Arduino UNO pins to the LEDs.
- Connect the DISPENSE and NEXT push buttons to their respective input pins.
- Connect the buzzer to the Arduino output pin.
- Establish common power and Ground (GND) connections.

Working Principle

The breadboard provides a common platform for assembling the circuit, while jumper wires establish the required electrical connections between the Arduino UNO and the hardware components.

Together, they make the prototype easier to assemble, test, troubleshoot and modify without requiring permanent circuit connections.

2.3.7 Medicine Storage Tray (four slot arrangement)

The medicine storage tray is the physical storage arrangement used in the Smart Medicine Sorting and Expiry Alert System (SMSES). The prototype consists of four individual slots, allowing medicines to be organized and monitored separately.

Each slot represents a medicine storage position in the system. The Arduino UNO maintains the corresponding medicine information and manages the slot status using the programmed data structures.

Features of the Storage Arrangement

1. **Four Individual Slots:** The tray provides four separate positions for storing medicine packages.
2. **Slot Identification:** Each slot is associated with a corresponding Green LED to indicate whether it is occupied.
3. **Expiry Priority Indication:** Red LEDs are used to indicate the slot containing the medicine with the highest FEFO priority.

4. **Inventory Management:** The system updates the slot information when a medicine is logically removed using the DISPENSE button.
5. **Circular Navigation:** The NEXT button allows navigation through the medicine slots using the Circular Doubly Linked List (CDLL).

Working Principle

When medicine details are entered into the system, the corresponding slot information is maintained by the Arduino UNO. The Green LED indicates an occupied slot, while the Red LED identifies the slot containing the medicine with the earliest expiry date.

When the DISPENSE button is pressed, the system removes the highest-priority medicine from its logical inventory and updates the corresponding slot indicators. The physical storage tray is used for organizing the medicine packages, while inventory removal is performed through the programmed system logic.

2.4 SOFTWARE DESCRIPTION

The software component of the Smart Medicine Sorting and Expiry System (SMSES) is developed using the Arduino IDE. It is responsible for processing medicine details, managing inventory records, implementing the FEFO (First-Expire-First-Out) principle, and controlling the connected hardware components.

The program is written in embedded C/C++ and runs on the Arduino UNO microcontroller. It uses data structures such as a Min-Heap, Circular Doubly Linked List (CDLL), and an array to manage medicine information and slot allocation.

Medicine details are entered through the Serial Monitor in a predefined format. The program extracts the required information, processes the expiry dates, and assigns medicines to the available slots. The Min-Heap identifies the medicine with the earliest expiry date, while the CDLL supports circular navigation between slots.

The software also handles user interactions through the DISPENSE and NEXT push buttons. It updates the Green and Red LEDs and controls the buzzer according to the system's programmed conditions.

The software is designed to operate within the memory and processing limitations of the Arduino UNO, making the prototype suitable for demonstrating data structures and embedded system programming.

2.4.1 Arduino IDE

Arduino IDE (Integrated Development Environment) is an open-source software platform used to write, compile, and upload programs to Arduino microcontroller boards. It provides a simple environment for developing and testing embedded system applications.

In the Smart Medicine Sorting and Expiry System (SMSES), Arduino IDE is used to develop and upload the program to the Arduino UNO. It also provides the Serial Monitor, which allows users to enter medicine details and interact with the system.

Features of Arduino IDE

1. **Source Code Editor:** Provides an editor for writing and modifying Arduino sketches with the .ino extension.
2. **Compiler:** Converts the written program into machine-readable instructions that can be executed by the microcontroller.
3. **Serial Monitor:** Enables serial communication between the computer and Arduino UNO. It is used to enter medicine details and view system messages.
4. **Board and Port Selection:** Allows the user to select the connected Arduino board and its corresponding communication port.
5. **Library Manager:** Provides access to additional libraries when required for project development.

Application in SMSES

Arduino IDE is used in this project for:

- Writing and editing the embedded C/C++ program.
- Compiling the program to identify syntax and compilation errors.
- Uploading the program to the Arduino UNO through a USB connection.
- Entering medicine information through the Serial Monitor.
- Testing button operations, LED indicators, and buzzer responses.

The Serial Monitor is configured at **115200 baud**, with a newline character used to indicate the end of each input entry.

Thus, Arduino IDE provides the programming and testing environment required to develop and operate the SMSES prototype.

2.4.2 Program Structure

The software of the Smart Medicine Sorting and Expiry System (SMSES) is organized into logical layers to manage hardware initialization, medicine data input, and system operations. The program runs on the Arduino UNO and uses serial communication to receive medicine details from the user.

The software structure consists of the following layers:

1. Initialization Layer

The initialization layer prepares the Arduino and connected hardware components before the system starts its normal operation.

It includes:

- Configuring the digital pins connected to the Green LEDs, Red LEDs, White LED, buzzer, and push buttons.
- Initializing the medicine record array.
- Initializing the Min-Heap and Circular Doubly Linked List (CDLL).
- Establishing serial communication at 115200 baud.
- Setting the initial states of the LEDs, including the White LED for power indication.
- Preparing the system to receive user inputs.

2. Input Handling Layer

The input handling layer receives medicine information through the Serial Monitor of the Arduino IDE.

The system accepts medicine details in the following format:

BATCH:201;EXP:YYYY-MM-DD;NAME:Paracetamol

The program performs the following operations:

- Reads incoming characters from the Serial interface.
- Stores the received characters in a temporary buffer.
- Detects the newline character to identify the completion of an input entry.
- Extracts the batch number, expiry date, and medicine name from the received data.
- Processes the extracted information for medicine record storage.

The input format is checked before the medicine information is processed. This helps prevent incorrectly formatted entries from being added to the inventory.

Overall Program Execution

After initialization and input processing, the program manages medicine records using the Min-Heap and CDLL. It also monitors the DISPENSE and NEXT buttons and updates the LEDs and buzzer according to the programmed conditions.

This layered structure makes the software easier to understand, test, and maintain while integrating data structures with embedded hardware.

2.4.3 Data Structures in Software

The Smart Medicine Sorting and Expiry System (SMSES) uses three main data structures: a Min-Heap, a Circular Doubly Linked List (CDLL), and an array of medicine records. These structures work together to manage medicine information, identify the earliest-expiring medicine, and support circular slot navigation.

1. Min-Heap (Priority Queue)

A Min-Heap is used to implement the First-Expire-First-Out (FEFO) principle. Each medicine is assigned a priority based on its expiry date. The medicine with the earliest expiry date is placed at the root of the heap.

- Maintains medicine priority according to expiry dates.
- The root element represents the medicine with the earliest expiry date.
- Provides O(1) time complexity for accessing the highest-priority medicine.
- Insertion and removal of the minimum element take O(log n) time.
- When a medicine is removed using the DISPENSE button, the heap is updated to identify the next earliest-expiring medicine.

2. Circular Doubly Linked List (CDLL)

A Circular Doubly Linked List is used to support circular navigation between medicine slots. Each node contains links to both the previous and next nodes, and the last node connects back to the first node.

- Supports movement in both forward and backward directions.
- Enables circular navigation without reaching a NULL end.
- Supports the NEXT button to move to the next medicine slot.
- Helps manage slot rotation and navigation when multiple medicines are stored.
- Can support tie-handling or rotation when medicines have the same expiry date, depending on the implemented logic.

3. Array of Medicine Records

An array is used to store the details of medicines entered into the system. It provides a simple way to access and manage the records associated with the four medicine slots.

Each medicine record contains the following information:

- Batch Number – Identifies the medicine batch.
- Medicine Name – Stores the name of the medicine.
- Expiry Timestamp – Represents the medicine's expiry date.
- Slot Assigned – Identifies the storage slot assigned to the medicine.
- Active Status – Indicates whether the medicine is currently present in the inventory.

The array provides direct access to medicine records using their indices and helps the system maintain a fixed number of medicine slots.

Conclusion

The Min-Heap manages expiry-based priority, the CDLL supports circular slot navigation, and the array stores medicine records. Together, these data structures provide an organized

way to manage medicine inventory and demonstrate the practical application of data structures in an embedded system.

2.4.4 Core Logic Modules

The software of the Smart Medicine Sorting and Expiry System (SMSES) is divided into different logical modules. Each module performs a specific task to manage medicine records, maintain expiry-based priority, control slot navigation, and provide user feedback.

1. Medicine Data Parsing Module

This module receives medicine information through the Arduino Serial Monitor. It extracts the required fields from the entered data.

- Accepts medicine details in a structured text format.
- Extracts the batch number, expiry date, and medicine name.
- Validates the entered data format.
- Converts the expiry date into a pseudo-timestamp for comparison.
- Stores the processed medicine information in the system.

2. FEFO Decision Module

This module identifies the medicine that has the earliest expiry date using the Min-Heap.

- Retrieves the earliest-expiring medicine from the root of the Min-Heap.
- Identifies the storage slot assigned to that medicine.
- Highlights the corresponding slot using the RED LED.
- Updates the priority indication whenever the inventory changes.
- Helps maintain the First-Expire-First-Out (FEFO) principle.

3. CDLL Rotation Module

This module manages circular navigation between the medicine slots using a Circular Doubly Linked List.

- Responds to the NEXT button.
- Moves the current pointer to the next slot.
- Supports continuous circular navigation.
- Helps manage navigation when multiple medicine slots are occupied.

4. Dispensing Module

This module is activated when the user presses the DISPENSE button.

- Identifies the highest-priority medicine using the Min-Heap.

- Removes the selected medicine from the logical inventory.
- Updates the corresponding medicine record and data structures.
- Turns OFF the Green LED of the cleared slot.
- Updates the Red LED to indicate the next highest-priority medicine.
- Provides an audio confirmation through the buzzer.

Note: In the current prototype, dispensing represents the logical removal of a medicine from the inventory. The system does not physically dispense medicine.

5. Near-Expiry Detection Module

This module checks the expiry information stored in the system and supports near-expiry alerts.

- Uses the stored expiry timestamp for comparison.
- Checks the highest-priority medicine for the programmed near-expiry condition.
- Activates the buzzer when the alert condition is met.
- Helps demonstrate expiry-warning functionality.

The current prototype uses simulated time for expiry calculations rather than an independent real-time clock.

6. Command Interpreter Module

This module supports additional commands through the Serial Monitor for testing and demonstration.

The system supports the following commands:

- **LIST:** Displays the stored medicine records and their statuses.
- **BEEP:** Tests the buzzer functionality.
- **FORCEALERT:** Triggers a simulated alert condition for demonstration.

These commands help test the system and observe its internal operations.

Conclusion

The core logic modules work together to process medicine details, identify expiry priority, navigate between slots, update inventory, and generate alerts. Their modular organization makes the software easier to understand, test, and maintain.

2.4.5 Timing & Real-Time Execution

The Smart Medicine Sorting and Expiry System (SMSES) uses a continuous execution loop to monitor user inputs, process medicine records, update LED indicators, and control the

buzzer. The Arduino UNO repeatedly executes the main program loop to ensure that the system responds to user actions.

1. Continuous Program Execution

The Arduino continuously performs the following operations:

- Reads medicine information entered through the Serial Monitor.
- Checks the DISPENSE and NEXT button states.
- Updates the Min-Heap according to medicine expiry priority.
- Updates the LED indicators based on slot occupancy and priority.
- Checks the programmed alert conditions.
- Activates the buzzer whenever an alert or confirmation is required.

2. Use of millis() Function

The Arduino millis() function provides the elapsed time in milliseconds since the microcontroller started running the program.

It can be used for:

- Managing time-based operations.
- Controlling periodic buzzer alerts.
- Implementing button debouncing.
- Avoiding unnecessary blocking delays.
- Scheduling operations within the main execution loop.

3. Pseudo-Timestamp for Expiry Management

The system converts the entered expiry date into a numerical pseudo-timestamp. This allows the Min-Heap to compare medicine expiry dates and identify the earliest-expiring medicine.

The pseudo-timestamp is used for expiry comparison and simulation. It is not obtained from an independent Real-Time Clock (RTC) module.

4. Button Debouncing

Button debouncing helps prevent multiple unwanted detections when a physical push button is pressed once.

A timing-based approach can be used to identify valid button presses and avoid repeated operations caused by contact bouncing.

5. System Response

The continuous execution of the program allows the system to respond to medicine entries and button operations while maintaining the correct LED and buzzer states.

Conclusion

The combination of continuous loop execution, timing functions, pseudo-timestamps, and button handling helps the Arduino UNO manage the system efficiently. These operations support responsive inventory management and expiry-priority demonstration.

2.4.6 Memory Optimization

The Arduino Uno has limited memory resources, including 2 KB of SRAM and 32 KB of Flash memory. Therefore, the Smart Medicine Sorting and Expiry System (SMSES) uses a lightweight software design to manage medicine records and perform priority-based operations efficiently.

The following approaches are considered for memory management:

1. **Fixed-Size Data Storage:** Fixed-size arrays and character buffers are used to store medicine details and process serial input.
2. **Efficient Data Structures:** A Min-Heap is used to manage medicine expiry priorities, while a Circular Doubly Linked List (CDLL) supports circular navigation between medicine slots.
3. **Limited Memory Usage:** The program avoids unnecessary data storage and excessive use of dynamic memory.
4. **Lightweight Processing:** The system uses simple operations and data structures suitable for the Arduino Uno's hardware limitations.
5. **Controlled Input Handling:** Medicine information is received and processed through the Serial Monitor using a structured input format.

These approaches help the prototype operate within the memory limitations of the Arduino Uno while supporting medicine record management, FEFO-based prioritization, slot navigation, and LED and buzzer control.

2.4.7 Testing and verification

The Smart Medicine Sorting and Expiry System (SMSES) was tested to verify the correct functioning of its hardware components, software modules, and data structures. Different test cases were performed to check medicine data entry, expiry-based prioritization, slot navigation, and alert generation.

The following tests were carried out:

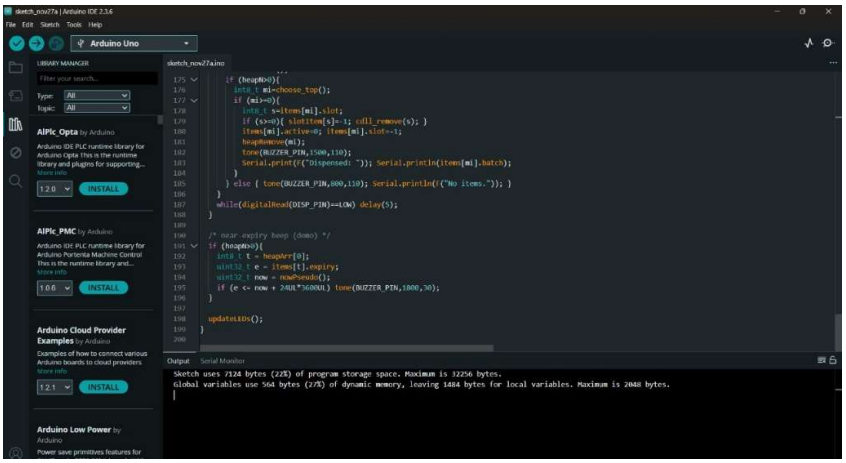
1. **Serial Input Testing:** Medicine details were entered through the Arduino Serial Monitor using the specified input format. The system was checked for correct data reception and parsing.
2. **Medicine Record Testing:** The LIST command was used to display the stored medicine records and verify the inventory information.

3. **FEFO Priority Testing:** The Min-Heap was tested to check whether the medicine with the earliest expiry date received the highest priority.
4. **LED Verification:** Green LEDs were checked to indicate occupied slots, while Red LEDs were checked to identify the medicine slot with the highest expiry priority. The White LED was verified as the power indicator.
5. **DISPENSE Button Testing:** The DISPENSE button was tested to verify the logical removal of the highest-priority medicine from the inventory and the corresponding update of the system status.
6. **NEXT Button Testing:** The NEXT button was tested to verify circular navigation between medicine slots using the CDLL.
7. **Buzzer Testing:** The BEEP and FORCEALERT commands were used to check the buzzer's audio output and programmed alert behavior.

Test Outcome

The prototype was evaluated through these functional tests to check the interaction between the Arduino Uno, software data structures, input buttons, LEDs, and buzzer. The tests helped identify and correct issues related to data entry, priority management, slot navigation, and output indications.

The testing demonstrates the working of the proposed FEFO-based inventory management prototype under the configured demonstration conditions.



CHAPTER – 3

PROJECT IMPLEMENTATION

3.1 WORKING

The Smart Medicine Sorting & Expiry System (SMSES) operates using an Arduino UNO, a Min-Heap, a Circular Doubly Linked List (CDLL), and LED and buzzer indicators. The system accepts medicine details through the Serial Monitor, processes the expiry information, assigns available slots, and identifies the medicine with the earliest expiry date using the FEFO (First-Expire-First-Out) principle.

The complete working procedure is explained below.

Step 1: Entering Medicine Details

The user enters medicine information through the Arduino Serial Monitor in the following format:

BATCH:XXX;EXP:YYYY-MM-DD;NAME:MedicineName

The input contains:

- Batch Number
- Expiry Date
- Medicine Name

The Serial Monitor is configured at 115200 baud with Newline as the line ending.

Step 2: Data Parsing and Storage

The Arduino receives the input and stores the characters in a temporary buffer. Once a complete line is received, the program extracts the medicine details.

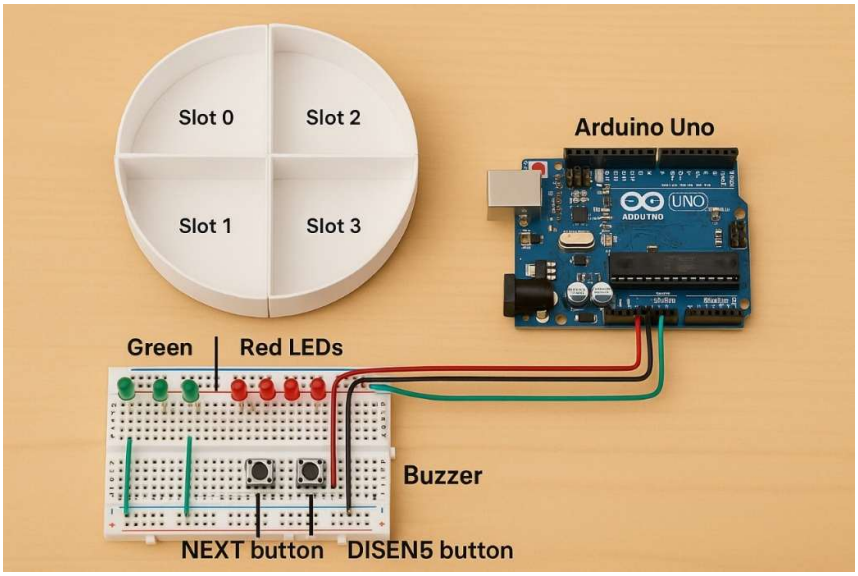
The extracted information is stored in an Item structure containing the medicine's batch number, name, expiry information, assigned slot, and active status.

The system checks the input format before processing the medicine record.

Step 3: Expiry Date Conversion

Since the base prototype does not use a Real-Time Clock (RTC) module, expiry dates are converted into pseudo-timestamps using the program's date-conversion logic.

These values allow the system to compare expiry dates and determine which medicine has the earliest expiry.



Step 4: Min-Heap Insertion (FEFO Logic)

After processing the medicine details, the system inserts the medicine into a Min-Heap.

The Min-Heap follows these operations:

- The root element represents the medicine with the earliest expiry date.
- New medicine records are inserted while maintaining the heap property.
- The earliest-expiry medicine can be identified from the root.
- When a medicine is dispensed, the corresponding heap entry is removed and the heap is updated.

This structure helps the system implement FEFO-based priority management efficiently.

Step 5: Circular Doubly Linked List (CDLL) Management

The system also uses a Circular Doubly Linked List to maintain circular navigation between medicine slots.

Its functions include:

- Maintaining a circular sequence of slots.
- Supporting forward and backward links between nodes.
- Allowing the NEXT button to advance the current CDLL position.
- Supporting rotation during demonstrations and equal-expiry handling.

The CDLL provides circular navigation, while the Min-Heap independently maintains expiry-based priority.

Step 6: LED Updates

The system uses nine LEDs to display its status:

- **Green LEDs:** Four Green LEDs indicate which medicine slots are occupied.
- **Red LEDs:** Four Red LEDs indicate the slot containing the highest-priority medicine.
- **White LED:** Indicates that the Arduino system is powered and active.

The LED states are updated according to the current medicine inventory and FEFO priority.

Step 7: DISPENSE Button Operation

When the user presses the DISPENSE button connected to pin D6:

1. The system identifies the highest-priority medicine using the Min-Heap.
2. The selected medicine is marked as inactive.
3. Its slot is released from the inventory.
4. The corresponding Green and Red LEDs are updated.
5. The buzzer produces a confirmation beep.

This operation represents the logical removal of the medicine from the inventory. Physical automatic dispensing is not implemented in the current prototype.

Step 8: NEXT Button Operation

When the user presses the NEXT button connected to pin D8:

- The CDLL head advances to the next position.
- The circular slot navigation is updated.
- The system provides a short buzzer sound as confirmation.

This feature demonstrates circular navigation using a Circular Doubly Linked List.

Step 9: FORCEALERT Function

The FORCEALERT command is provided as a demonstration feature.

When entered through the Serial Monitor, it modifies the expiry simulation of the priority medicine so that the near-expiry alert condition can be demonstrated without waiting for an actual expiry date.

This makes it easier to demonstrate the alert mechanism during project presentations and evaluations.

Step 10: Near-Expiry Detection and Buzzer Alert

The Arduino continuously checks the expiry status of the highest-priority medicine using the program's pseudo-time logic.

When the medicine falls within the configured 24-hour near-expiry threshold:

- The buzzer produces an alert tone.
- The Red LED indicates the priority medicine.

- The system provides an audio indication to draw the user's attention.

This feature demonstrates how expiry-based alerts can support better medicine inventory management.

3.2 RESULTS

The Smart Medicine Sorting & Expiry System (SMSES) was implemented using an Arduino UNO and tested with sample medicine records. The prototype demonstrated the integration of data structures, embedded programming, and hardware components for expiry-based medicine inventory management.

The following results were observed during testing:

1. Accurate Medicine Data Handling

Medicine details entered through the Serial Monitor were processed by the Arduino. The system extracted the batch number, medicine name, and expiry date and stored the information for further processing.

2. FEFO Priority Selection

The Min-Heap successfully identified the medicine with the earliest expiry date. This demonstrated the implementation of the FEFO (First-Expire-First-Out) principle using a priority queue.

3. Effective Visual Feedback

The LED indicators provided information about the system status:

- Green LEDs indicated occupied medicine slots.
- Red LEDs highlighted the highest-priority medicine.
- The White LED indicated system power status.

4. Successful CDLL Navigation

The NEXT button was used to demonstrate circular navigation through the medicine slots using the Circular Doubly Linked List (CDLL).

5. DISPENSE Operation

The DISPENSE button triggered the logical removal of the highest-priority medicine from the inventory. The corresponding slot status and LED indicators were updated, and the buzzer provided confirmation.

6. Buzzer and Alert Testing

The BEEP command was used to test the buzzer. The FORCEALERT command was used to demonstrate the near-expiry alert mechanism using simulated expiry information.

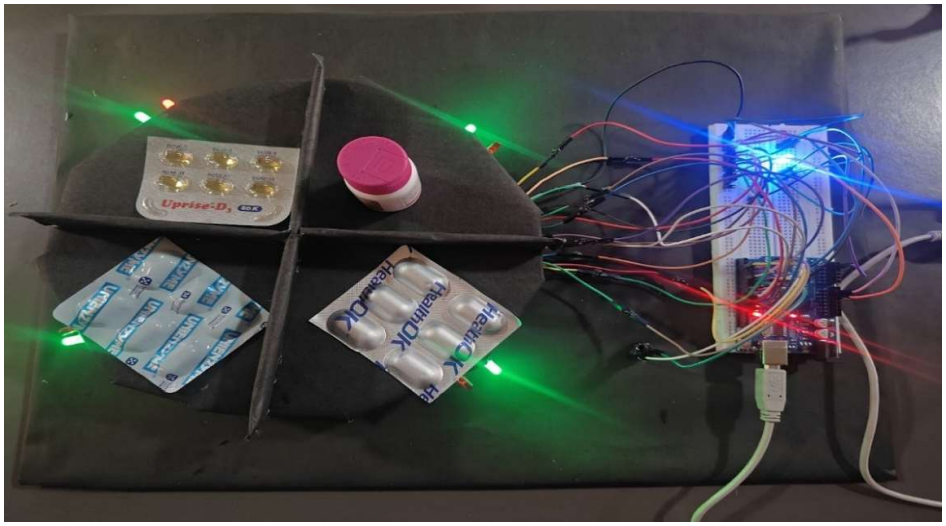
7. Inventory Display

The LIST command displayed the stored medicine information, including batch number, slot number, active status, and days-to-expiry information.

Overall Result

The prototype demonstrated the working of a Min-Heap for expiry-based prioritization and a Circular Doubly Linked List for circular slot navigation. It also showed how an Arduino-based embedded system can combine software logic with physical indicators and user controls to manage a small medicine inventory.

The developed system successfully served as an academic demonstration of data structures and embedded system concepts applied to medicine inventory management.



3.3 ADVANTAGES

The Smart Medicine Sorting and Expiry System (SMSES) offers several advantages by combining data structures with a simple Arduino-based hardware setup.

1. **Expiry-Aware Medicine Management:** The system uses the First-Expire-First-Out (FEFO) principle to identify and prioritize medicines with the earliest expiry dates. This helps users manage medicine stock more effectively and reduces the possibility of overlooking medicines nearing expiry.
2. **Low-Cost Implementation:** The prototype uses affordable and easily available components, such as an Arduino Uno, LEDs, push buttons, a buzzer, and a breadboard. This makes it suitable for academic demonstrations and learning purposes.
3. **Practical Application of Data Structures:** The project demonstrates the use of a Min-Heap for expiry-based prioritization and a Circular Doubly Linked List (CDLL) for circular slot navigation. It connects theoretical data structure concepts with a practical hardware application.

4. **Simple User Interface:** LEDs provide visual indications of slot status and priority, while push buttons allow users to navigate slots and perform logical dispensing operations. The buzzer provides an audible alert for near-expiry conditions.
5. **Modular Design:** The system is designed so that additional features, such as barcode scanning, automatic dispensing, and environmental monitoring, can be considered in future versions.
6. **No External Sensors Required:** The basic prototype operates without additional environmental sensors. Medicine information is entered through the Arduino Serial Monitor and processed by the programmed logic.
7. **Expandable Inventory Management:** Although the current prototype supports four medicine slots, its design provides a foundation for developing systems with a larger number of slots and additional inventory-management features.

3.4 DISADVANTAGES

Although the Smart Medicine Sorting and Expiry System (SMSES) demonstrates an effective approach to expiry-based medicine management, the current prototype has certain limitations.

1. **Manual Data Entry:** Medicine details, including batch number, expiry date, and medicine name, must be entered manually through the Arduino Serial Monitor. This may lead to incorrect entries if the information is not provided accurately.
2. **No Physical Automatic Dispensing:** The current prototype supports logical dispensing by updating the medicine inventory when the dispense button is pressed. However, it does not physically dispense or release medicines from the tray.
3. **Limited Storage Capacity:** The prototype currently supports only four medicine slots. Therefore, its storage capacity is limited and may not be sufficient for managing a large medicine inventory.
4. **No Integrated Barcode Scanner:** The basic version does not include a physical barcode scanner. Medicine information is entered through the Serial Monitor rather than being scanned automatically.
5. **No Real-Time Clock (RTC):** The current implementation uses a software-based pseudo-time mechanism for expiry comparisons instead of a dedicated Real-Time Clock module. Therefore, it cannot independently maintain accurate calendar time.
6. **Prototype-Level Implementation:** The system is designed primarily for academic demonstration and data structure implementation. Further development, testing, and validation would be required before considering practical deployment in pharmacies, hospitals, or other healthcare environments.

CHAPTER 4

CONCLUSION

4.1 Conclusion

The Smart Medicine Sorting and Expiry System (SMSES) was developed as an Arduino-based prototype to demonstrate expiry-aware medicine inventory management using data structures. The primary objective of the project was to implement the First-Expire-First-Out (FEFO) principle and prioritize medicines based on their expiry dates.

The system uses a **Min-Heap** to identify the medicine with the earliest expiry date and a **Circular Doubly Linked List (CDLL)** to support circular navigation between medicine slots. These data structures demonstrate how concepts learned in Data Structures and Algorithms can be applied to a practical hardware-based application.

The Arduino Uno acts as the main controller and processes medicine information entered through the Serial Monitor. LEDs provide visual indications of medicine slot status and priority, push buttons support slot navigation and logical dispensing operations, and a buzzer provides audible alerts for near-expiry conditions.

The prototype successfully demonstrates the integration of software logic, data structures, and basic electronic components in a low-cost embedded system. It also provides practical experience in programming, hardware interfacing, data processing, and problem-solving.

Overall, the project establishes a foundation for developing more advanced medicine inventory management systems. With further improvements, features such as barcode scanning, real-time clock integration, automatic dispensing, and environmental monitoring could be incorporated to enhance its functionality.

4.2 Future Scope

The Smart Medicine Sorting and Expiry System (SMSES) can be further enhanced by incorporating additional hardware components and software features. The following improvements can increase its functionality, automation, and usability.

1. **Barcode Scanner Integration:** A barcode scanner can be added to automatically read medicine information, reducing manual data entry and the possibility of input errors.
2. **Real-Time Clock (RTC) Module:** An RTC module, such as the DS3231, can be integrated to maintain accurate date and time information. This would improve expiry-date monitoring without depending on software-based pseudo-time.
3. **Automatic Medicine Dispensing:** A servo motor or suitable mechanical dispensing mechanism can be incorporated to enable physical medicine release based on the priority identified by the system.
4. **Environmental Monitoring:** Temperature and humidity sensors can be added to monitor the storage environment. This would help identify conditions that may affect medicine storage requirements.

5. **Mobile Application Integration:** A mobile application can be developed to display medicine details, expiry information, slot status, and alerts, providing users with a more convenient interface.
6. **Cloud-Based Inventory Management:** Cloud integration can enable medicine inventory records to be stored and accessed remotely. It could also support notifications and inventory monitoring across multiple devices.
7. **Increased Storage Capacity:** The prototype can be extended to support more medicine slots by modifying the hardware arrangement and updating the data structure implementation.
8. **Enhanced Data Security:** Authentication and secure data storage mechanisms can be introduced to protect medicine records and prevent unauthorized modifications.

These enhancements can transform the current academic prototype into a more advanced medicine inventory management system. Each feature would require further development, integration, and testing before practical deployment.

BIBLIOGRAPHY

[1] Arduino, *Arduino Language Reference*, Arduino Documentation.

[2] Arduino, *Arduino UNO Rev3 – Hardware Documentation*, Arduino Documentation.

[3] Arduino, *Arduino IDE – Software Documentation*, Arduino Documentation.

[4] Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, *Fundamentals of Data Structures in C*, Universities Press.

[5] Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd Edition, Prentice Hall, 1988.

[6] Arduino, *Digital Input and Output, Serial Communication and Timing Functions*, Arduino Documentation.

[7] Smart System Designing course materials and laboratory notes, SR University.

APPENDIX

APPENDIX A: HARDWARE PIN CONFIGURATION

The following table represents the pin connections used in the Smart Medicine Sorting and Expiry System (SMSES) prototype.

S. No.	Component	Arduino Pin
1	Dispense Push Button	D6
2	Next Push Button	D8
3	Buzzer	D7
4	Green LED – Slot 1	D10
5	Red LED – Slot 1	D11
6	Green LED – Slot 2	D12
7	Red LED – Slot 2	D13
8	Green LED – Slot 3	D4
9	Red LED – Slot 3	D5
10	Green LED – Slot 4	D2
11	Red LED – Slot 4	D3
12	White Power Indicator LED	A4

APPENDIX B: MEDICINE DATA INPUT FORMAT

Medicine information is entered manually through the Arduino Serial Monitor.

The Serial Monitor is configured with a baud rate of **115200** and uses a newline character to identify the end of each input.

The medicine information follows the format:

BATCH:201;EXP:2025-01-05;NAME:Paracetamol

The input fields are described below:

1. **BATCH:** Represents the medicine batch number.
2. **EXP:** Represents the medicine expiry date in YYYY-MM-DD format.
3. **NAME:** Represents the name of the medicine.

The entered information is parsed by the Arduino program and stored for further processing.

APPENDIX C: SERIAL MONITOR COMMANDS

The system supports the following commands through the Arduino Serial Monitor:

S. No.	Command	Description
1	LIST	Displays the stored medicine inventory information.
2	BEEP	Activates the buzzer for a short duration.
3	FORCEALERT	Simulates a near-expiry condition for demonstration purposes.

These commands help users interact with the prototype and verify its basic functionalities.

APPENDIX D: DATA STRUCTURES USED

The following data structures are implemented in the project:

1. Min-Heap:
Used to prioritize medicines according to their expiry dates. The medicine with the earliest expiry date is maintained at the root of the heap.

2. Circular Doubly Linked List (CDLL):
Used to maintain circular navigation between the four medicine slots. Each node represents a medicine slot and supports forward and backward navigation.

3. Structure Array:
Used to store medicine-related information, including batch number, medicine name, expiry timestamp, slot number, and active status.

4. Static Arrays:
Used for maintaining medicine records, heap information, slot mapping, and circular linked-list connections.

APPENDIX E: ARDUINO SOURCE CODE

The complete Arduino source code used in the implementation of the Smart Medicine Sorting and Expiry System is included in this section.

The program contains the following major modules:

1. Hardware pin configuration and initialization.
2. Medicine data input and parsing.
3. Min-Heap insertion, deletion, and priority management.
4. Circular Doubly Linked List initialization, insertion, deletion, and navigation.
5. LED status indication.
6. Push-button input handling.
7. Buzzer alert generation.
8. Serial Monitor command processing.
9. Medicine inventory display and logical dispensing.

The complete source code is attached below for reference and verification.

```
#define DISP_PIN 6
#define NEXT_PIN 8
#define BUZZER_PIN 7

#define G0 10
#define R0 11
#define G1 12
#define R1 13
#define G2 4
#define R2 5
#define G3 2
#define R3 3
#define WHITE_LED A4

#define SLOTS 4
#define MAX_ITEMS 8

typedef struct { uint16_t batch; char name[14]; uint32_t expiry; int8_t slot; uint8_t active; } Item;
static Item items[MAX_ITEMS];
static int8_t slotItem[SLOTS];

static int8_t heapArr[MAX_ITEMS];
static int8_t heapPos[MAX_ITEMS];
static uint8_t heapN = 0;
static uint8_t itemCount = 0;

static int8_t cdll_next[SLOTS];
static int8_t cdll_prev[SLOTS];
static uint8_t cdll_in[SLOTS];
```

```
static int8_t cdll_head = -1;


static char inBuf[96];
static uint8_t inPos = 0;


static inline uint32_t toTS(const char *s){
  uint16_t y = (s[0]-'0')*1000 + (s[1]-'0')*100 + (s[2]-'0')*10 + (s[3]-'0');
  uint8_t mo = (s[5]-'0')*10 + (s[6]-'0');
  uint8_t d  = (s[8]-'0')*10 + (s[9]-'0');
  return 1600000000UL + (uint32_t)(y - 2000) * 365UL * 86400UL + (uint32_t)(mo - 1) *
31UL * 86400UL + (uint32_t)(d - 1) * 86400UL;
}
static inline uint32_t nowPseudo(void){ return (millis()/1000UL) + 1600000000UL; }
static inline uint32_t keyu(uint8_t i){ return items[i].expiry; }


/* heap */
static void hswap(uint8_t a,uint8_t b){ int8_t t=heapArr[a]; heapArr[a]=heapArr[b];
heapArr[b]=t; heapPos[heapArr[a]]=a; heapPos[heapArr[b]]=b; }
static void heapUp(uint8_t i){ while(i){ uint8_t p=(i-1)/2; if
(keyu(heapArr[i])>=keyu(heapArr[p])) break; hswap(i,p); i=p; } }
static void heapDown(uint8_t i){ for(;;){ uint8_t l=2*i+1,r=2*i+2,s=i; if (l<heapN &&
keyu(heapArr[l])<keyu(heapArr[s])) s=l; if (r<heapN &&
keyu(heapArr[r])<keyu(heapArr[s])) s=r; if (s==i) break; hswap(i,s); i=s; } }
static void heapPush(int8_t idx){ heapArr[heapN]=idx; heapPos[idx]=heapN;
heapUp(heapN); heapN++; }
static void heapRemove(int8_t idx){ int8_t p=heapPos[idx]; if (p<0) return; heapN--;
heapArr[p]=heapArr[heapN]; heapPos[heapArr[p]]=p; heapPos[idx]=-1; heapDown(p);
heapUp(p); }


/* CDLL for slots */
static void cdll_init(void){ for(int8_t i=0;i<SLOTS;i++){ cdll_in[i]=0;
cdll_next[i]=cdll_prev[i]=-1; } cdll_head=-1; }
static void cdll_insert(int8_t s){ if (s<0||s>=SLOTS) return; if (cdll_in[s]) return; if
(cdll_head==-1){ cdll_head=s; cdll_next[s]=s; cdll_prev[s]=s; cdll_in[s]=1; return; } int8_t
```

```
t=cdll_prev[cdll_head]; cdll_next[t]=s; cdll_prev[s]=t; cdll_next[s]=cdll_head;
cdll_prev[cdll_head]=s; cdll_in[s]=1; }

static void cdll_remove(int8_t s){ if (s<0||s>=SLOTS) return; if (!cdll_in[s]) return; if
(cdll_next[s]==s){ cdll_in[s]=0; cdll_head=-1; cdll_next[s]=cdll_prev[s]=-1; return; } int8_t
p=cdll_prev[s], n=cdll_next[s]; cdll_next[p]=n; cdll_prev[n]=p; if (cdll_head==s)
cdll_head=n; cdll_in[s]=0; cdll_next[s]=cdll_prev[s]=-1; }

static int8_t cdll_find_exp(uint32_t e){ if (cdll_head==-1) return -1; int8_t cur=cdll_head;
do{ int8_t mi = slotItem[cur]; if (mi>=0 && items[mi].active && items[mi].expiry==e)
return cur; cur = cdll_next[cur]; } while(cur!=cdll_head); return -1; }

static int8_t cdll_advance(void){ if (cdll_head==-1) return -1; cdll_head =
cdll_next[cdll_head]; return cdll_head; }


/* core */

static int8_t freeSlot(void){ for(int8_t i=0;i<SLOTS;i++) if (slotItem[i]==-1) return i; return -
1; }


static void printList(void){
  Serial.println(F("---- LIST ----"));
  uint32_t now = nowPseudo();
  if (itemCount==0){ Serial.println(F("No stored items.")); return; }
  for(uint8_t i=0;i<itemCount;i++){
    Serial.print(i); Serial.print(F(": B=")); Serial.print(items[i].batch);
    Serial.print(F(" S=")); Serial.print(items[i].slot);
    Serial.print(F(" A=")); Serial.print(items[i].active);
    Serial.print(F(" D="));
    if (!items[i].active){ Serial.println(F("N/A")); continue; }
    if (items[i].expiry <= now){ Serial.println(F("EXPIRED")); continue; }
    Serial.println((items[i].expiry - now) / 86400UL);
  }
}


/* Force-alert: make current top expire soon (now + 10s) */
static void doForceAlert(void){
```

```
  if (heapN == 0) { Serial.println(F("No items to force.")); return; }
  int8_t topIdx = heapArr[0];
  items[topIdx].expiry = nowPseudo() + 10; // 10 seconds until expiry
  Serial.print(F("FORCEALERT applied to batch ")); Serial.println(items[topIdx].batch);
}

/* parse barcode or LIST/BEEP/FORCEALERT */
static void parseLine(const char *line){
  if (!line) return;
  if (strncmp(line,"LIST",4)==0){ printList(); return; }
  if (strncmp(line,"BEEP",4)==0){ tone(BUZZER_PIN,1500,200); return; }
  if (strncmp(line,"FORCEALERT",10)==0){ doForceAlert(); return; }

  if (itemCount >= MAX_ITEMS){ Serial.println(F("Storage full")); return; }
  char t[96]; char exp[12]=""; char nm[14]=""; int batch=-1;
  strncpy(t,line,sizeof(t)-1); t[sizeof(t)-1]=0;
  char *tok = strtok(t,";");
  while(tok){
    if (!strncmp(tok,"BATCH:",6)) batch=atoi(tok+6);
    else if (!strncmp(tok,"EXP:",4)) strncpy(exp,tok+4,11);
    else if (!strncmp(tok,"NAME:",5)) strncpy(nm,tok+5,sizeof(nm)-1);
    tok=strtok(NULL,";");
  }
  if (batch<0 || strlen(exp)<10){ Serial.println(F("Invalid barcode")); return; }
  int8_t id = itemCount++;
  items[id].batch=(uint16_t)batch;
  strncpy(items[id].name,nm,sizeof(items[id].name)-1);
  items[id].expiry=toTS(exp);
  items[id].active=1;
  items[id].slot=freeSlot();
```

```
  if (items[id].slot>=0){ slotItem[ items[id].slot ] = id; cdll_insert(items[id].slot); }
  else Serial.println(F("No free slot"));
  heapPos[id]=-1; heapPush(id);
  Serial.print(F("Added: ")); Serial.print(batch); Serial.print(F(" Slot: "));
Serial.println(items[id].slot);
}

static int8_t choose_top(void){ if (heapN==0) return -1; int8_t top=heapArr[0]; uint32_t
e=items[top].expiry; int8_t s = cdll_find_exp(e); if (s>=0) return slotItem[s]; return top; }

static void updateLEDs(void){
  digitalWrite(G0,LOW); digitalWrite(R0,LOW);
  digitalWrite(G1,LOW); digitalWrite(R1,LOW);
  digitalWrite(G2,LOW); digitalWrite(R2,LOW);
  digitalWrite(G3,LOW); digitalWrite(R3,LOW);
  digitalWrite(WHITE_LED,HIGH);
  if (slotItem[0]!=-1) digitalWrite(G0,HIGH);
  if (slotItem[1]!=-1) digitalWrite(G1,HIGH);
  if (slotItem[2]!=-1) digitalWrite(G2,HIGH);
  if (slotItem[3]!=-1) digitalWrite(G3,HIGH);
  if (heapN>0){
    int8_t mi = choose_top();
    if (mi>=0){
      int8_t s = items[mi].slot;
      if (s==0) digitalWrite(R0,HIGH);
      else if (s==1) digitalWrite(R1,HIGH);
      else if (s==2) digitalWrite(R2,HIGH);
      else if (s==3) digitalWrite(R3,HIGH);
    }
  }
}
```

```
/* setup & loop */
void setup(){
  Serial.begin(115200);
  pinMode(DISP_PIN, INPUT_PULLUP);
  pinMode(NEXT_PIN, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(G0,OUTPUT); pinMode(R0,OUTPUT);
  pinMode(G1,OUTPUT); pinMode(R1,OUTPUT);
  pinMode(G2,OUTPUT); pinMode(R2,OUTPUT);
  pinMode(G3,OUTPUT); pinMode(R3,OUTPUT);
  pinMode(WHITE_LED,OUTPUT); digitalWrite(WHITE_LED,HIGH);
  for(int8_t i=0;i<SLOTS;i++) slotItem[i]=-1;
  for(int8_t i=0;i<MAX_ITEMS;i++) heapPos[i]=-1;
  cdll_init();
  Serial.println(F("SMSES 4-slot optimized ready. Paste barcodes or type
LIST/BEEP/FORCEALERT"));
}

void loop(){
  while(Serial.available()){
    char c=(char)Serial.read();
    if (c=='\r') continue;
    if (c=='\n' || inPos >= (int)sizeof(inBuf)-2){ inBuf[inPos]=0; if (inPos>0) parseLine(inBuf);
inPos=0; }
    else if (inPos < (int)sizeof(inBuf)-1) inBuf[inPos++]=c;
  }

  /* NEXT button */
  static unsigned long lastN=0;
  if (digitalRead(NEXT_PIN)==LOW){
```

```
  if (millis()-lastN>180){ lastN=millis(); int8_t h=cdll_advance(); if (h>=0){
tone(BUZZER_PIN,1200,70); Serial.print(F("NEXT->")); Serial.println(h); } }
  while(digitalRead(NEXT_PIN)==LOW) delay(5);
 }

 /* DISPENSE */
 static unsigned long lastD=0;
 if (digitalRead(DISP_PIN)==LOW){
  if (millis()-lastD>220){
   lastD=millis();
   if (heapN>0){
    int8_t mi=choose_top();
    if (mi>=0){
     int8_t s=items[mi].slot;
     if (s>=0){ slotItem[s]=-1; cdll_remove(s); }
     items[mi].active=0; items[mi].slot=-1;
     heapRemove(mi);
     tone(BUZZER_PIN,1500,110);
     Serial.print(F("Dispensed: ")); Serial.println(items[mi].batch);
    }
   } else { tone(BUZZER_PIN,800,110); Serial.println(F("No items.")); }
  }
  while(digitalRead(DISP_PIN)==LOW) delay(5);
 }

 /* near-expiry beep (demo) */
 if (heapN>0){
  int8_t t = heapArr[0];
  uint32_t e = items[t].expiry;
  uint32_t now = nowPseudo();
  if (e <= now + 24UL*3600UL) tone(BUZZER_PIN,1800,30);
```

```
  }

  updateLEDs();
}
```